Predicting LLM Performance on Chess Puzzles using Public Data

Sam Infante, Noah Kim, Jaeho Lee, Reid Povinelli

Brown University

GitHub: https://github.com/reidpov/DATA1030_midterm.git

Introduction

AI companies release new models quickly, often faster than evaluators can test them. When a new model is released, there is little information available about independent benchmarks until weeks later. However, running a benchmark has high costs, as Epoch AI's Chess Puzzles benchmark lets each model work until it reaches a budget of 1,000,000 output tokens.[1] In this project, the team will attempt to predict a model's score on Chess Puzzles v2.0.0 using publicly available information about the model. A good score prediction would be useful to several stakeholders, and can also give insights into the benchmark itself. If its scores can be predicted accurately from public information, the benchmark may not add much information on the model's performance.

The team chose to focus on the chess model because each puzzle has one correct answer that can be checked against a chess engine for accuracy. Solving the puzzle requires thinking multiple moves ahead in time, and Epoch describes the benchmark as, "100 novel puzzles, generated programmatically with a chess engine," and notes that the positions "do not appear in any other source."[2] This shows that the model cannot simply memorize the answers.

Other researchers have tried to predict how well AI models will do on benchmarks. Owen (2024) studied how performance scores change as models are trained with more computing power. He stated, "average benchmark performance is decently predictable as a function of training compute scale."[3] There has also been work done with chess and AI, with Kagi testing language models on 1,000 puzzles taken from a public chess domain.[4] However, since these puzzles were public, the models may have seen them during training. Epoch's benchmark avoids this by using newly generated puzzles.

Data

Our benchmark results and model metadata were sourced from Epoch AI's Benchmarking Hub. Epoch AI is a research organization that tracks AI progress, and the hub's data is available as a public Airtable database under a CC BY 4.0 license. We made a copy of the public Airtable base on October 20, 2026 and downloaded four of its tables through the Airtable API using the `pyairtable` Python library. The tables are linked by Airtable record IDs, which we kept to use as join keys.

Table	Rows	Columns	Row Description
Tasks	25	6	Each row is a benchmark

[1] https://epoch.ai/benchmarks/chess-puzzles
[2] https://epoch.ai/benchmarks/chess-puzzles
[3] https://arxiv.org/abs/2401.04757
[4] https://github.com/kagisearch/llm-chess-puzzles

Runs	2,267	22	Each row is one evaluation/run of a benchmark
Model Versions	1,169	10	A specific model setting (e.g. gpt-5.4-high)
ML Models	3,725	53	A model on its own, without any settings

The columns are largely of three types: categorical (e.g. country), numeric (e.g. compute in FLOP), or dates (e.g. release date).

In addition to the EpochAI we looked for supplemental data to make a more comprehensive model given the sparsity of features. We evaluated multiple candidate data sources including but not limited to OpenRouter, Artificial Analysis (AA), and Hugging Face APIs for more information on the models. We ultimately chose OpenRouter as the sole addition for model metadata. The benefit of OpenRouter was that it introduced a broad set of additional features and matched well with basic base model matching logic, while the matching for AA and Hugging Face was fuzzier and the features not as consistent.

The OpenRouter catalog supplements Epoch AI's sparse model metadata with context-window size, supported modalities, reasoning settings, token pricing, knowledge-cutoff dates, and, for some models, AA and Design Arena ratings. We retrieve a snapshot from the OpenRouter API using urllib to send GET requests and matched records by matching normalized base-model names, with version names used when a base-model match was unavailable. While this results in matching at the base-model level and not the version level, we are able to retain version-specific details such as reasoning level by extracting it from the name and turning it into features as separate columns. The combined dataset contains a host of categorical, numeric, date, text, list, and boolean fields.

Our target is a model version's score on Epoch AI's Chess Puzzles benchmark, which asks a model to identify the best move in each chess position. The score is the fraction of puzzles answered correctly, from 0 to 1. The raw Runs table contains 392 Chess Puzzles evaluations. We selected the 148 successful evaluations using benchmark version 2.0.0, then retained the 140 that could be matched to OpenRouter metadata. Each row in the resulting dataset represents one model version and its benchmark run. Scores range from 0 to 0.74.

The final dataset has 140 rows and 156 columns, but many of these columns are there simply for completeness sake and will be dropped before prediction. A model version's release date will allow us to train on earlier models and use later models as a test set but also introduces some extra complexity as we deal with temporal features and non-stationary data.

In total, 137 features were dropped. 22 features were not available at time of deployment, 58 were mostly empty, 9 were constant, 19 were IDs or other non-useful text, 12 were high

cardinality, 9 were redundant, and 8 were dropped for other specific reasons. More information on the dropping strategy can be found in the code. After dropping, we were left with 140 rows and 19 columns (not including target variable).

Exploratory Data Analysis

After unused features were dropped, the remaining features were sorted into categories based on data type; continuous, continuous dates, categorical, and ordinal. Exploratory univariate figures were created to visualize each of the remaining features, with histograms for continuous features and bar charts for categorical features. Next, bivariate figures were created to visualize each feature against the target variable: scatter plots for continuous features and box plots for categorical features. Additionally, the Spearman correlation was calculated for each numerical feature and plotted in a bar chart, demonstrating that the AA Intelligence, Coding, and Agentic Indices were the three variables with the highest correlation, in that order. Finally, explanatory figures were created to demonstrate important takeaways.

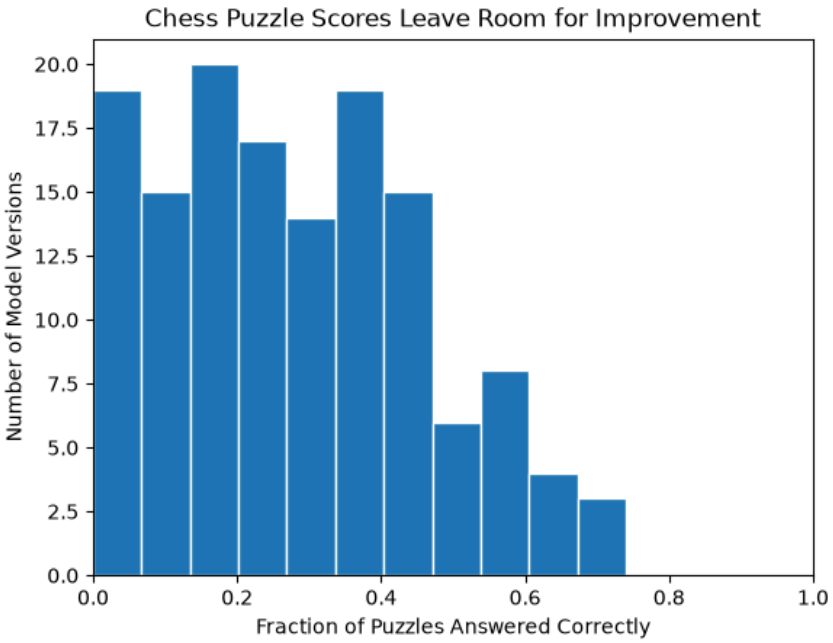


Figure 1: Distribution of Chess Score

Figure 1 is a histogram of Chess Puzzle v2 scores across 140 model versions, showing that the scores range from 0 to 0.74, with most values below 0.5. This means that this benchmark is not saturated, so the differences between models are meaningful and worth predicting.

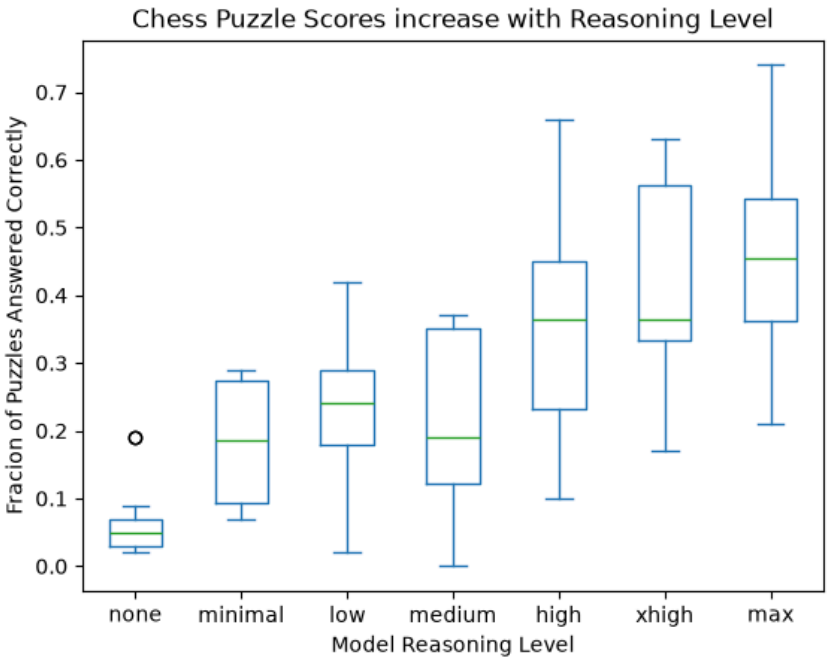


Figure 2: Chess Score by Reasoning Level

Figure 2 shows box plots of Chess Score for each reasoning level ordered by increasing reasoning effort. The median values rise with effort, but there is still some spread within each level. This is important because reasoning level is the only feature that differs between versions of the same model, so it is the main source of information about within-model differences.

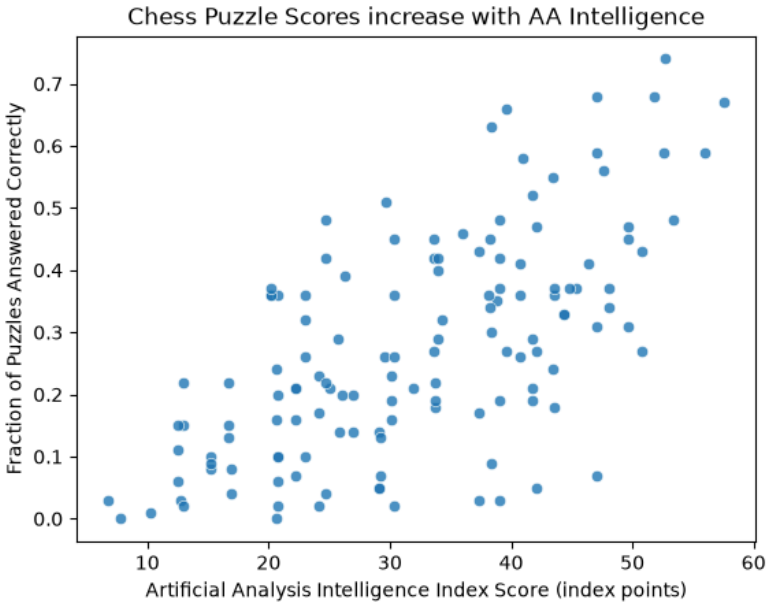


Figure 3: AA Intelligence Index vs. Chess Score

Figure 3 is a scatter plot that shows a clear positive relationship between each model's AA Intelligence Index and its Chess Score. Interestingly, because every version of a model gets the same index, there are visible vertical stacks at specific x-values. This spread reflects the reasoning-level difference shown in Figure 2. This is important because features like this can provide real information about Chess Score performance, however cannot distinguish between versions of the same model, so the kinds of features shown in Figure 2 and 3 are both needed.

Problem description and potential impact

Our problem is predicting how well a LLM will score on Epoch AI's benchmark Chess Puzzles v2.0.0 before the benchmark is actually run. Each row of our feature matrix is a model version with its reasoning setting: for instance, GPT-5.4 at high effort. Its features are descriptive of the model, including developer, release date, and context length. They also include scores on third-party leaderboards, reported through OpenRouter, including AA's benchmark aggregates and Design Arena ratings. The target variable is the fraction of puzzles that the model is able to solve. Because the target is continuous (and happens to be bounded between 0 and 1), this is a regression problem.

Our goal is to forecast the benchmark scores of models that haven't been run on Chess Puzzles v2.0.0 yet. Given that most models are run on Chess Puzzles as they are released, we split the data by release date, and intend to train on earlier models while testing on later ones.

As for impact: running Chess Puzzles properly is expensive, as the agentic harness runs until a budget of one million tokens is reached. Although this is considered relatively "cheap" to well-resourced benchmarking organizations, it's still a nontrivial cost (e.g. $1000-2500 for GPT-6 Astra) and an independent evaluator might be interested in getting a prediction before committing funds to a full benchmark run. Not only that, a model developer could use them to get a preliminary estimate of how new models or reasoning settings might perform before formally testing it.

References

Artificial Analysis. *Artificial Analysis*. https://artificialanalysis.ai/ (accessed October 2026).

Epoch AI. *AI Benchmarking Hub* (CC BY 4.0). https://epoch.ai/benchmarks (accessed October 20, 2026).

Epoch AI (2025). *Chess Puzzles*. https://epoch.ai/benchmarks/chess-puzzles (accessed October 2026).

Hugging Face. *Hugging Face Hub*. https://huggingface.co/ (accessed October 2026).

OpenRouter. *OpenRouter API Reference*. https://openrouter.ai/docs/api_reference/ (accessed October 2026).

Owen, D. (2024). How predictable is language model benchmark performance? arXiv:2401.04757. https://arxiv.org/abs/2401.04757

Prelovac, V. (2024). *LLM Chess Puzzles*. Kagi, GitHub repository. https://github.com/kagisearch/llm-chess-puzzles

pyAirtable. *Python client for the Airtable API*. https://github.com/gtalarico/pyairtable